Lab 1 – Programming in Assembly & Embedded C

Embedded Processors UESTC2004 & UESTCHN2004 School of Engineering, University of Glasgow

Student Name: Chen Yichi

Student Number: [redacted]

UESTC 2004 Lab Assessment Criteria

Each individual lab will be evaluated using the below criteria:

- Demonstrating a functional embedded system and lab questions – 70%

  This means each student will need to demonstrate a functional system for each task in the lab manual to the GTA and member of staff to get the full mark.

- Reflection and significance – 5%

  After the completion of programming and testing each of the "Tasks", you will be required to reflect, in a few lines, on your understanding of the tasks and its significance. This is a chance for you to ask yourselves "What you did?", "What it means?", and "What next?", meaning what is the significance of the undertaken task in the context of embedded systems and what you are learning in the lecture. Try to be innovative and think outside the box. Please, do not provide a summary of what you did as it will not be accepted.

- Adherence to coding good practice – 25%
  - Meaningful variable names – please avoid names like "x", "y", or personal names
  - Using block comments to explain what your code does at the beginning and using line comments to clarify what this line of code does.
  - Consistent indentation – Indentation helps in making the code readable and easy to interpret. See an example of what to do and what to avoid, below:

Good practice ✓

```
#include "mbed.h"
char switch_word ;         //the word we will send
char recd_val;            //the received value
int main() {
     async_port.baud(9600);      //set baud rate to 9600
```

```
      recd_val = 0;
      while (1)
      {
         switch_word=0xa0;
      }
}
```

Avoid this ✗

```
#include "mbed.h"
char switch_word ;        //the word we will send
char recd_val;           //the received value
int main() {
async_port.baud(9600);           //set baud rate to 9600
recd_val = 0;
while (1)
{
switch_word=0xa0;
}
}
```

All the best of luck.

The Embedded Processors Team,

---

1. Lab Objective

This lab includes two tasks. The objective of the first one "Task 1" is to practice coding in assembly language and understand registers and various instruction sets used while coding. This lab also aims to familiarise you with the development board, breadboard, and writing C programs for the NUCLEO L432KC mbed board. In "Task 2" you will implement your first simple embedded system in the course on the NUCLEO L432KC board.

2. Requirements

Software

Task 1:

- Download and install the ARMSim#1.91 from Moodle-> Lab Material -> "Lab 1 - Introdction to Assembly"(Folder).
- N.B.: To run ARMSim# 1.91 you must be running a **Windows system** with a **.NET** Framework

Task 2:

- mbed online compiler (https://ide.mbed.com/compiler/#nav:/;) – You will need to create an account to use the online compiler.

3. Tasks

Task 1: Programming in Assembly

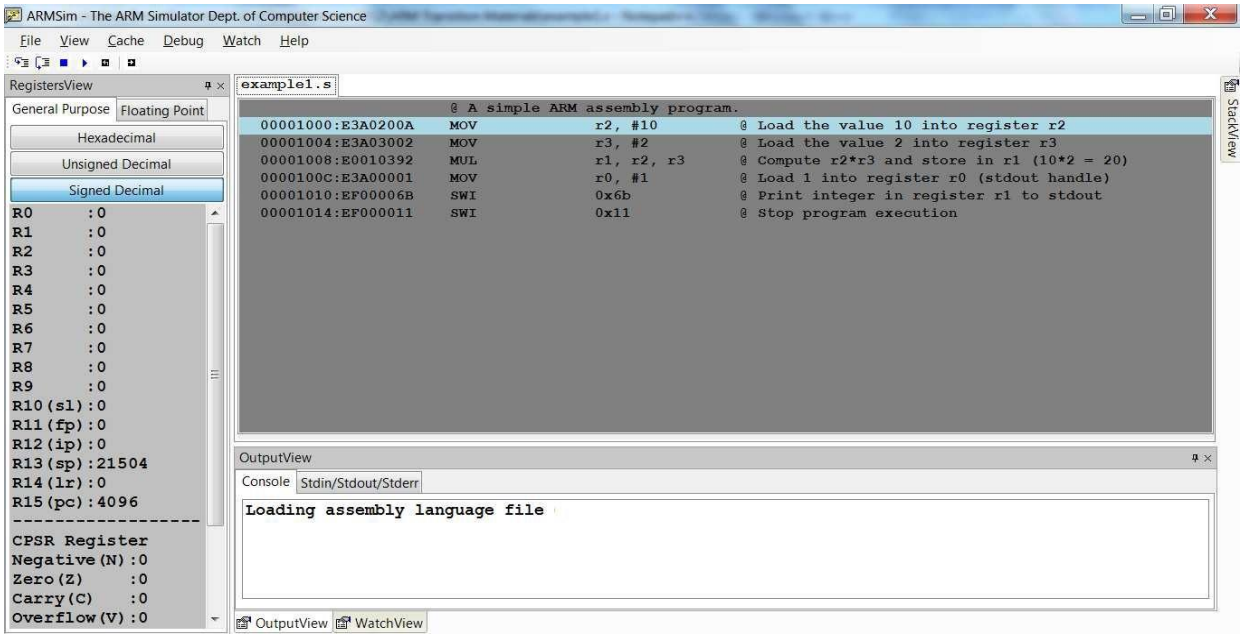
Task 1.1: ARMSim# is not an editor, so you will need to create and edit your ARM assembly program with a separate text editor (such as Notepad++). Open your favourite text editor and type in the following simple program:

Do not worry if some of these instructions are unfamiliar at the moment, they will be covered in time. Save your program as example1.s – note the '.s' file extension is required in order for ARMSim# to read your file. Once you have saved your ARM program, you need to load it into ARMSim#. Open the ARMSim# program, select 'File -> Load' from the menu, and select your example1.s file. It should open your program and make sure it is without syntactic errors.
Your screen will look like the one below. Your example1.s file should look like below:

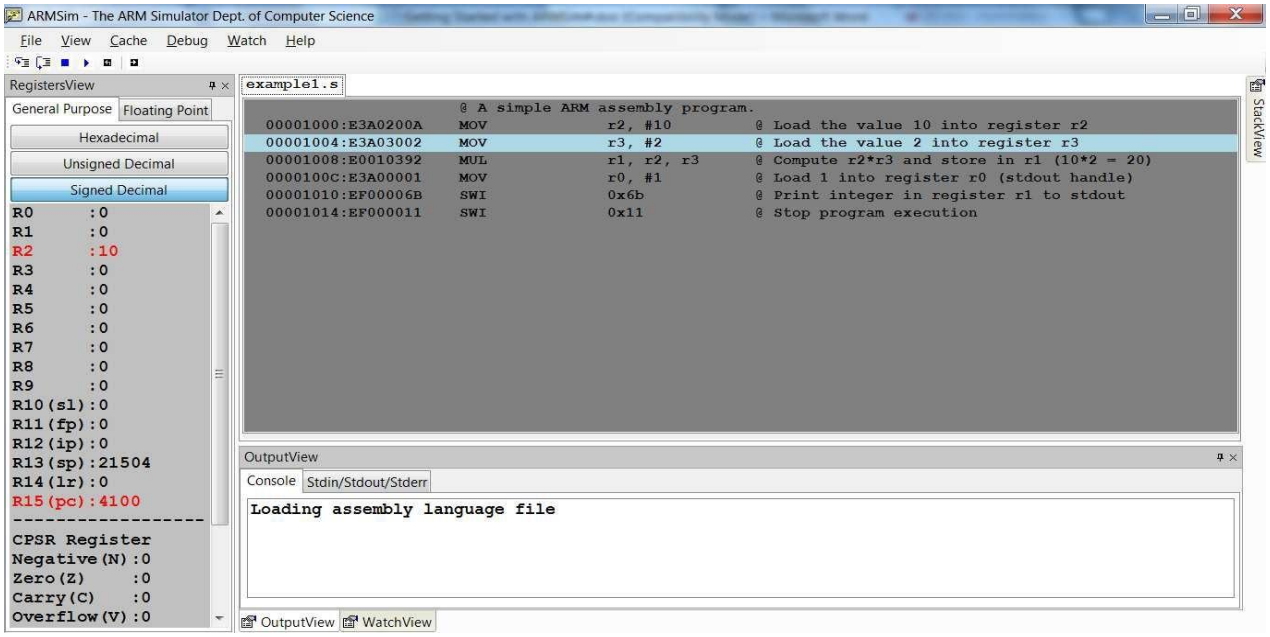
```
@ A simple ARM assembly program.
MOV r2, #10 @ Load value 10 into register r2
MOV r3, #2 @ Load value 2 into register r3
MUL r1, r2, r3 @ Compute r2*r3 & store in r1(10*2 = 20)
MOV r0, #1 @ Load 1 into register r0 (stdout handle)
SWI 0x6b @ Print integer in register r1 to stdout
SWI 0x11 @ Stop program execution
```

On the left side of the screen, the registers for the simulated ARM processor are displayed. The source code window is in the center, while the console window is at the bottom.

There are three options for running (simulating) your assembly language program – run, step into, and step over. The two that will likely be of most use (and their toolbar icons) are (i) run( ) and (ii) step into ( ). Run (keyboard shortcut F5) will run your assembly language program from start to finish, while step into (keyboard shortcut F11) will execute one instruction at a time, allowing you to see how each instruction is modifying the state.



Press “Step Into”, and you should see a screen like the one below.



On the left hand side, you will see that the registers that have been changed by the executed instruction are red. In the source code window, you will notice that the highlighted blue line is now the next instruction – is the instruction pointed to by the program counter, and will be executed next. Try stepping through the remainder of the simple program, verifying that the registers are changing as expected.

Task 1.2: Complete the following tasks with reference to the codes in Task 1.1

Write a program to perform the subtraction of a value in register 1 from a value in register 2 and the result should be stored in register 0. Once completed, show the demo to the GTA to get your mark.

```
@ A simple ARM assembly program.
MOV r1, #10
MOV r2, #2
SUB r0, r1, r2
SWI 0x11
```

Task 1.3: Write a program to multiply the value in register 1 with the value in register 2 and store the result in register 0.

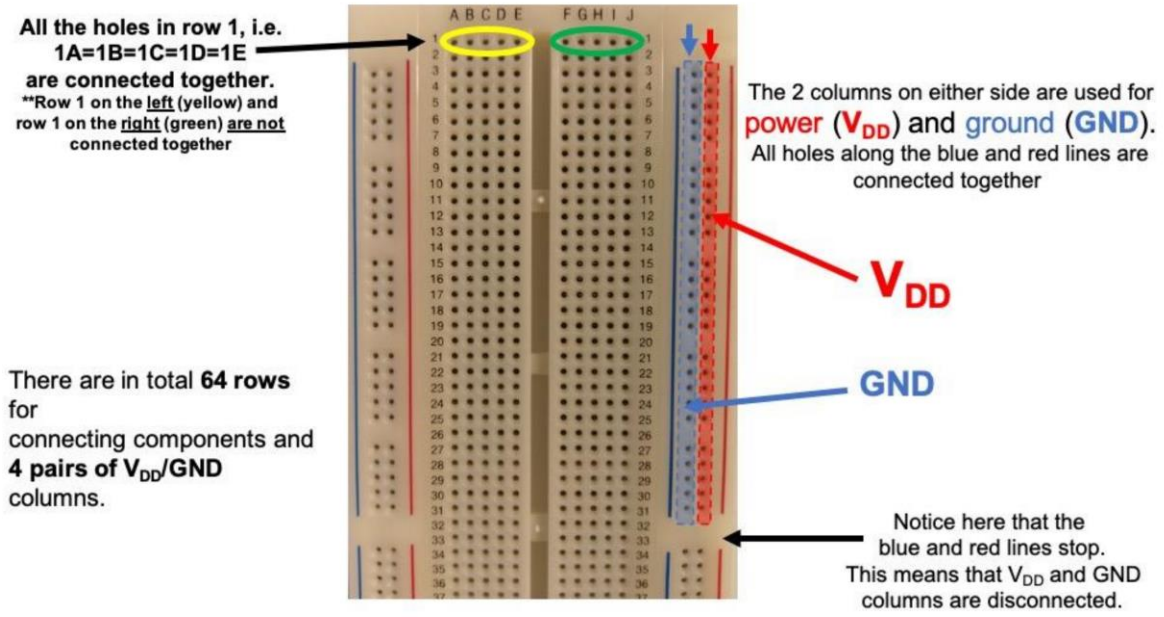
```
MOV r1, #4096
MOV r2, #4096
MUL r0, r1, r2
SWI 0x11
```

Task 1.4: With the multiplication program, set registers 1 and 2 to 4096. What is the result from the simulator? **[Answer in the below space]**



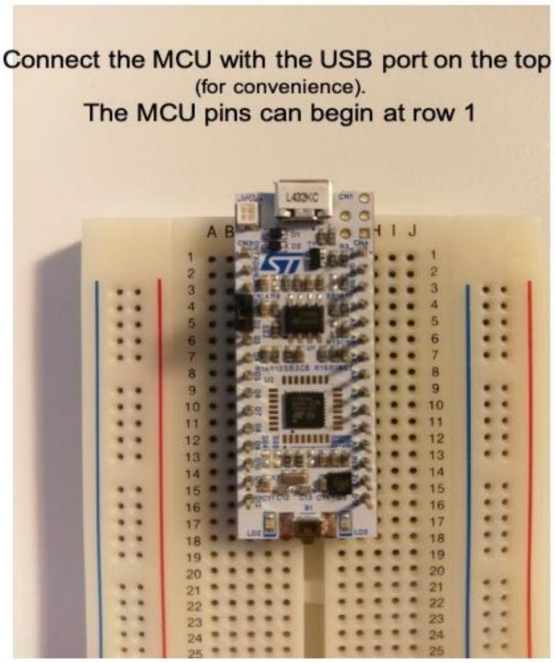
Getting Started with Your Development Breadboard

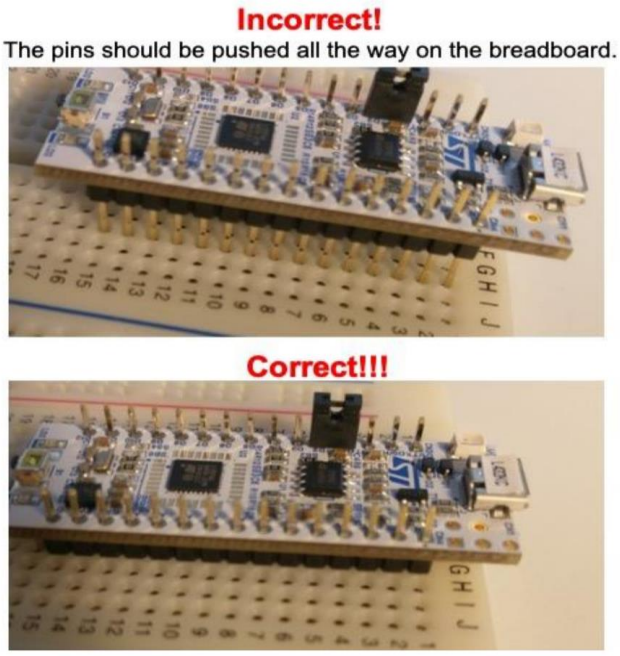
The breadboard that you will be using will look like the one in the picture below, it is a standardised component and particularly useful for developing prototype circuits.



Please study the picture above and understand how a Breadboard works!!

****Mount your microcontroller like is shown in the picture below.**

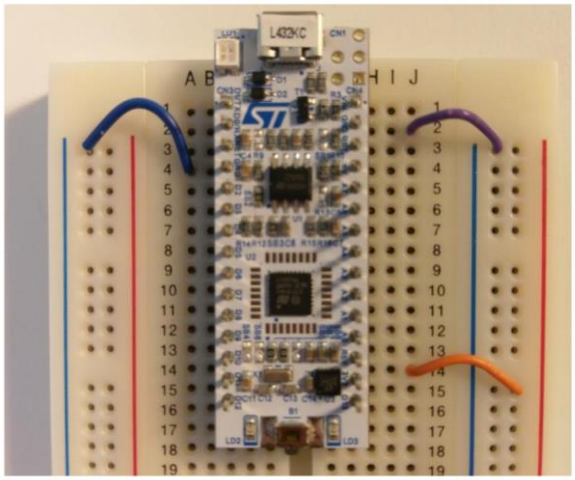




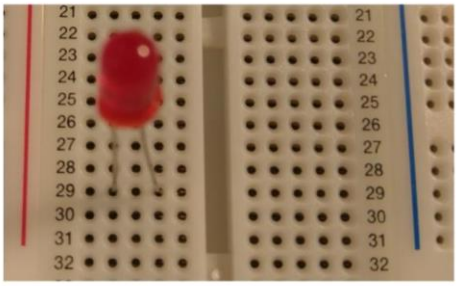
Before creating any complicated prototypes on our breadboard, the very first thing to do is

to connect the VDD (red) and GND (blue) columns with our microcontroller. By doing so at the beginning, we can create tidy prototypes that can be easily debugged if there are any problems. On your MCU there are two GND pins and one VDD pin (3.3V).

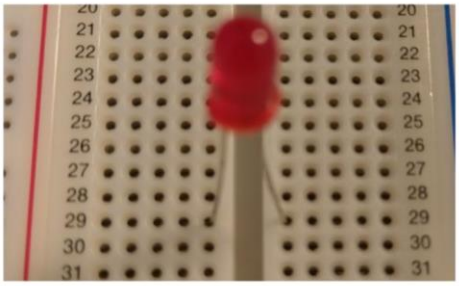
Use wires with appropriate colours to define GND and VDD



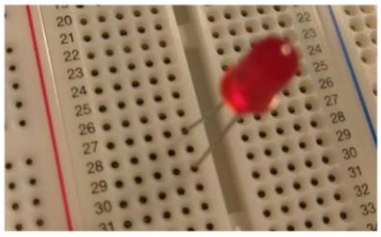
How to connect components on your Breadboard



Incorrect! Both LED legs are connected on the same row, therefore connected together



Correct! Both LED legs are connected on the same row, but on columns E and F. Therefore not connected together.



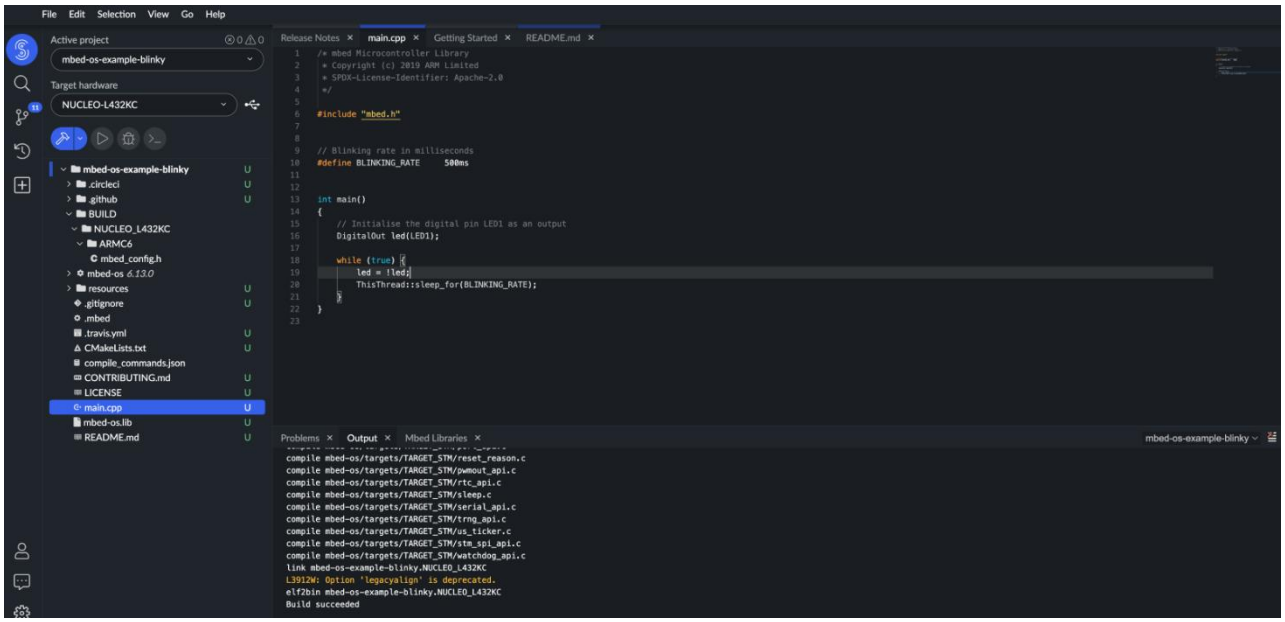
Correct! Both LED legs are connected on different rows. Therefore not connected together.

Getting Started with the Programming

First, you need to get yourself an mbed account. Go to the armKeil, https://www.keil.arm.com/, and follow the instructions on getting started (https://www.keil.arm.com/mbed/?utm_source=mbed_com&utm_medium=compiler_button&utm_campaign=mbed_com_promo).
You need to have your mbed connected to your computer via the USB cable. The red LD2 should be lit.

You need to tell the compiler what target board you have. This is in the top left of the compiler screen.

You can now create a new program to check everything is working. Click New in the menu bar at the top of the page, on the top left of the compiler pane. It will be called “mbed-os-example-mbed5-blinky” by default. Click OK. The compiler screen looks like the figure below.



To get the program onto the mbed, you first have to compile it, by clicking the "Compile" button in the toolbar  . This will create the binary code which will ultimately be downloaded to the mbed. If the compiling goes well, you will get a "Success!" message in the compiler output, and a popup will prompt you to download the compiled .bin file to the mbed. Your computer won’t know what to do with this file type, but the microcontroller will.

```
Problems 2   Output   Mbed Libraries
scan /tmp/chroots/ch-09c00cc8-83bb-4d40-ad28-c3848fc000f7/extras/mbed-os.lib
link mbed-os-example-blinky.NUCLEO_L432KC
compile main.cpp
L3912W: Option 'legacyalign' is deprecated.
elf2bin mbed-os-example-blinky.NUCLEO_L432KC
Build succeeded


Flash: 31 KB / 256 KB
Memory: 64 KB / 64 KB
ROM: 32 KB
```

The file "*mbed-osexample-mbed5 blinky.NUCLEO_L432KC.bin*" is what want to copy over USB to the MCU board. On your computer, the microcontroller board comes up as a USB drive called *NODE_L432KC*. While the program is downloading, the status LED on the mbed (*LD1*) should flash quickly. Your mbed will run the newest program you send to it.

Once you write a program to the Nucleo board via USB, it will run immediately. The normal status is that *LD1* is green and *LD2* is red. This example program flashes the green *LD3* on and off. For this particular program, you should see *LD3* flashing on and off every 0.5 seconds.

```c
/* mbed Microcontroller Library
 * Copyright (c) 2019 ARM Limited
 * SPDX-License-Identifier: Apache-2.0
 */

#include "mbed.h"

#define WAIT_TIME_MS 500
DigitalOut led1(LED1);

int main()
{
    while (true)
    {
        led1 = !led1;
        thread_sleep_for(WAIT_TIME_MS);
    }
}
```

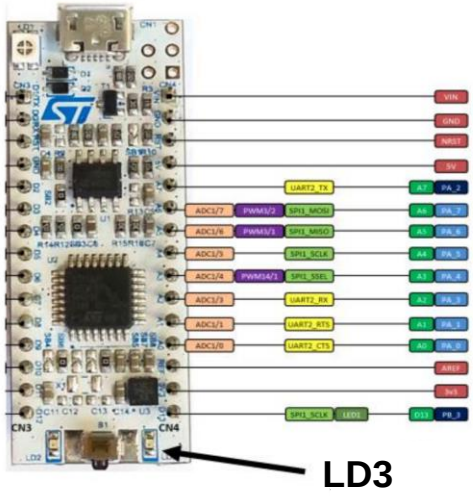


Figure Mbed-os-example-blinky Code

To make more programs, just click New to create a new program, and enter a filename. Your newly created program will appear in the Program Workspace on the left of the screen. If you expand the program by clicking on the >, you will see something called main.cpp - double click on this, it will open your program in the compiler environment.

As you can see, it only takes a few lines of code to program the mbed. This is because of the Application Programming Interface (API) that was mentioned previously. Basically, this is a set of programming building blocks, which are C++ utilities, that allow rapid code development. You can find links to the various library blocks from the mbed website handbook page. You can follow links for each library function that describes the programming syntax as well as giving some examples to make things more obvious.

Commenting on C/C++

Line comment: A line that starts with // is a comment and is ignored by the compiler. Comments are there to explain how the program is working, and help greatly when you need to submit code for assessment or when working as a team.

Multi-line comment: All the lines between /* and */ are coloured in grey and are ignored by the compiler.

The program:

#include "mbed.h"

this is a header that needs to be at the start of each mbed program – it is a link to a set of libraries specific to your hardware – in other words it tells the generic C++ compiler all the important information about the target of the program (your STM32L432KC) and contains all the 'standard' mbed functions

#define WAIT_TIME_MS 500

This is a great way to have numbers that you use in the program in one place at the start. When you need to change the numbers, you only need to look at the top of the code rather than wading through thousands of lines. Everywhere you use the name *WAIT_TIME_MS* in your program, it is replaced by the compiler with the number 500 before compiling. 500 milliseconds is 0.5seconds (there are 1000 milliseconds in a second).

int main() {

the first action of any C++ program is contained within its **main()** function. The function definition, i.e., what goes on inside the function - is contained within the curly brackets, starting immediately after **main()**, and continuing until the last closing curly bracket } (line 18). On the microcontroller you have, the main function runs only once every time you press the reset button – it does not repeat. This is not particularly useful – imagine a lift that went to one floor and then stopped forever. We need its code to keep running and running…

DigitalOut led(LED1);

essentially, what you are doing here is using the **DigitalOut** component (from the mbed.h library) to set **LED1, i.e. *LD3*,** as a digital output, and to give that "port" a handle or name, **led**. You can do something similar for each of the other Digital IO pins, using the format DigitalOut name(pinnumber); e.g. DigitalOut greenled(D1) would set pin D1 as a digital output, with the name greenled That's the definitions over, so now next we get on to the actual program that will run

while(true) {

many embedded systems programs contain an endless loop, ie a program that just goes on repeating. In other branches of programming, this is bad practice, but for embedded systems, this is necessary. The endless loop is created using the **while** keyword; this

controls the code within the curly brackets which follow. Normally, while is used to set up a loop, which repeats only if a certain condition is satisfied, but if we write **while(true)**, or **while(1)** this will make the loop repeat endlessly. The value 1 is the same as Boolean 'true', which means that we have asked while the value is true do the stuff in brackets. Be careful, C++ is case sensitive, so true in lower case is not the same to the compiler as TRUE in capitals.

The part of the program that actually causes the LED to switch on and off is contained in the lines within the **while** loop. There are two calls to the library function thread_sleep_for(), and a statement in which the value of **led** is changed.

Led1 = 1;
means that the variable **led** is set to the value 1, whatever its previous value was. In C++, this is different to "equals", which is represented by "= =". "=" has the meaning "is set to". This sets a logic 1, or voltage of 3.3V on the pin to which LED1 is connected. This will cause the LED to light up since its cathode is connected to ground.

led1 = !led1;
means that the variable **led** is set to a value that is opposite of its previous value. The ! means logical NOT (or invert the Boolean value). This sets a logic 1, or voltage of 3.3V on the pin to which LED1 is connected if it was previously logic 0 or OV, and vice versa. This will cause the LED to light up (and turn off) since its cathode is connected to ground, in an alternating fashion as the Boolean value 'led' goes 1 0 1 0 1 0 1 0...

thread_sleep_for(WAIT_TIME_MS);
function is from the mbed library - you can find out more about it on the mbed website. In previous mbed versions this command used to be simply wait(). Nowadays, the wait command is not the preferred one since there are altenrative low power options that do not use the CPU, but let it sleep while the timer peripheral is running. Since the variable in the parenthesis is immediately replaced by the value 500, this is effectively thread_sleep_for(500). The 500 parameter is a 32bit integer in milliseconds, and defines the delay length caused by this function. So the LED will be lit for 0.5s. You can also use smaller units for the wait, e.g. a wait of 200 milliseconds is the same as 0.2s, and write (200).

NOTE: The thread_sleep_for( ) takes inputs that are integer numbers. So if you want to use 1 microsecond (1us), you should use the commands,
wait_us(x) → sleep for x microseconds
wait_ns(x) → sleep for x nanoseconds
The following commands will be depreciated in the future,
wait(x) → sleep for x seconds

wait_ms(x) → sleep for x milliseconds

The program will then reach the } on line 17. The commands in the while-loop are over, and so the program proceeds to start the while again, and because its an infinite loop while(true) so the LED flashing will continue forever…

You can explore the syntax for many of the mbed libraries by following links from the mbed handbook homepage. http://mbed.org/handbook/Homepage

Task 2: LED Brightness Control with Push Buttons

Objective: Design and implement an embedded system using the Nucleo L432KC board and Mbed that allows users to control the brightness of an LED using push buttons. One button will increase the brightness, and the other button will decrease it. Show the demo to the GTA.

What you will need:

- 1 x Nucleo L432KC development board
- 1 x LED
- 2 x Push buttons
- Resistors (if required for LED current limiting and/or for push buttons to pull up or down)
- Breadboard and jumper wires

Setup:

- Connect an LED to the Nucleo L432KC board's pin.
- Connect two push buttons to the pins for brightness control.

Initialisation:

- Define a PWM output for controlling LED brightness (led1).
- Define digital inputs for the two push buttons (button and button2).

Main Program:

- Implement a loop to continuously monitor the state of the push buttons.
- When one button is pressed, set a flag to indicate the desired action (increase or decrease brightness).
- Adjust LED brightness gradually based on the button press:
    - If the first button is pressed, increase brightness gradually.
    - If the second button is pressed, decrease brightness gradually.
    - If no button is pressed, toggle the LED on/off.

Gradual Brightness Change:

- Use a loop to increment or decrement the LED brightness gradually.
- Use *thread_sleep_for()* to introduce delays for a smooth brightness transition.

Testing:

- Test the functionality by pressing the buttons and observing the LED brightness changes.
- Verify that the LED turns on/off when no button is pressed.

Documentation:

- Document the circuit connections, code explanation, and any observations during testing.
- Take note of any challenges faced and how they were resolved.

[Paste your code below after you show the demonstration to the GTA and use the space to write any information about your design if needed]

```
#include "mbed.h"

PwmOut led1(D9);     // PWM output for LED brightness control
DigitalIn button(D11,PullDown);    // First push button
DigitalIn button2(D10,PullDown);    // Second push button

int main()
{
    bool incr = false;
    bool decr = false;
    float brightness = 0.0f;
    int delay_ms = 1000;
    led1.period_ms(1);/set the period to be 1ms
    while (1)
    {
        if (button == 1)
        {
            incr = true;
        }
        else if (button2 == 1)
        {
```

```
    decr = true;
}
else // No button pressed, toggle LED on/off
{

    led1.write(brightness);
    thread_sleep_for(250);
    led1.write(0.0f);
    thread_sleep_for(250);



}

if (incr==true) //increase brightness
{
    if (brightness < 1.0)
    {
        brightness += 0.1f;
        led1.write(brightness);/set the duty cycle to be 0.5
        thread_sleep_for(delay_ms);
    }
    incr = false;
}

if (decr==true) //decrease brightness
{
    if (brightness > 0.0)
```

```
            {
                brightness -= 0.1f;
                led1.write(brightness);/set the duty cycle to be 0.5
                thread_sleep_for(delay_ms);
            }
            decr = false;
        }
    }
}
```

4. Reflection

After the completion of programming and testing of Tasks 1 and 2 please reflect, in a few lines, on your understanding of the tasks and its significance. This is a chance for you to ask yourselves **"What you did?", "What it means?", "What next?", and/or How else, more efficiently, can this be done?,** meaning what is the significance of the undertaken task in the context of embedded systems and what you are learning in the lecture. Try to be innovative and think outside the box. **Please, do not provide a summary of what you did as it will not be accepted.**

[Answer in the below space]

Task1：I used ARMsim to do some simple operation: assignment,subtraction, multiplication .
I also found that numbers which can be written as power of two could ignore the boundary of digit places.
Task2: I wrote a mbed code to enable using two buttons. One acted as signal to increase the brightness of the LED. Another acted as signal to ddecrease the brightness of the LED.

5. Lab Score

Marking Criteria	Student's Score	Comments from the GTA
Demonstration Task 1 **Task 1.1 – Not graded** **Task 1.2 – 15%** **Task 1.3 – 15%** **. Task 1.4 – 5%**		
. Demonstration Task 2 (35%)		
Reflection (5%)		
Adherence to coding good practice (25%)		